

Tema 3 — Identidad y Autenticación de Agentes de IA

Identidad y Autenticación de Agentes de IA — Índice

1. Sin identidad verificable, la seguridad colapsa
2. OAuth y SAML no fueron diseñados para agentes
3. El problema de bootstrapping
4. El error del "candy shell"
5. Cuatro fases del ciclo de vida de identidad
6. Los bloques de una Zero Trust Architecture
7. Más allá de un solo trust domain
8. IAM tradicional es peligrosamente inadecuado

3.1 Sin identidad verificable, la seguridad colapsa

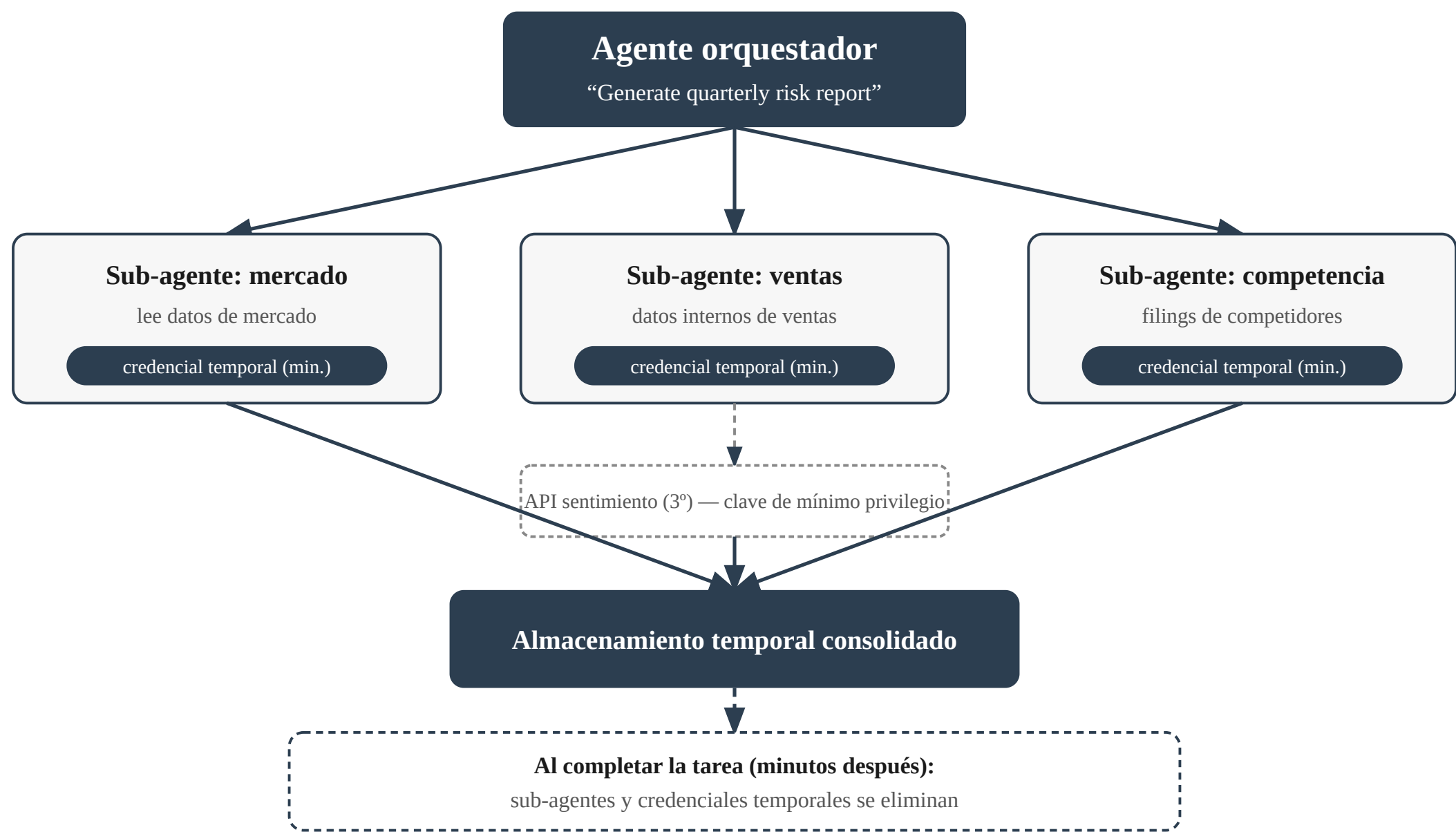
- Toda autorización, logging y auditoría depende de responder: "**¿quién hace esta acción?**"
- Sin certeza criptográfica sobre el "quién", la postura de seguridad se derrumba
- **IA Agéntica**: software que persigue metas complejas de forma autónoma (planificación + razonamiento + tool use)
- Introduce una **crisis de identidad** que los sistemas legacy no anticiparon (Google Cloud, 2025; Benson, 2024)
- A diferencia de un usuario humano o un servidor con IP estática: el agente es dinámico, efímero, impredecible

3.1 Un agente financiero, minutos de autonomía

Meta: *"Generate a quarterly risk assessment report"* — el agente, sin supervisión:

- Levanta 3 sub-agentes en paralelo (mercado, ventas internas, filings de competidores)
- Otorga acceso temporal de solo lectura a tablas y APIs internas
- Usa una API de sentimiento de terceros con una clave específica
- Consolida resultados en almacenamiento temporal
- Elimina sub-agentes y credenciales al terminar

Fig. — Flujo del agente orquestador y sus sub-agentes efímeros



3.1 Las preguntas que el IAM tradicional no responde

- ¿Cómo se emiten, gestionan y revocan credenciales para sub-agentes efímeros que viven minutos?
- ¿Cómo se garantiza que la API de sentimiento use la clave de mínimo privilegio y no la "god-mode" de un developer?
- ¿Cómo se genera un log auditable que pruebe qué agente hizo qué acción exacta?
- El **IAM tradicional** —piedra angular empresarial por décadas— está mal equipado para este paradigma (Huang, 2025; TechTarget, 2025)
- Ruta del tema: diseccionar fallas del IAM → construir **autenticación** (probar quién es) → extender a **autorización** (decidir qué puede hacer)

3.2 OAuth y SAML no fueron diseñados para agentes

- **OAuth 2.0** y **SAML 2.0**: caballos de batalla de la identidad digital por décadas
- Sobresalen en su propósito original: usuarios humanos interactuando con apps web
- Aplicados a comunicación M2M/agent-to-agente dinámica y autónoma → fallas críticas
- Asegurar identidades no humanas exige ir más allá de la gobernanza de acceso tradicional (Huang et al., 2024a)

3.2 SAML autentica, OAuth autoriza

SAML — Autenticación

- XML-based, habilita SSO empresarial
- IdP (Okta, Azure AD) firma una "Assertion" → Service Provider (Cantor et al., 2005)
- "Yo autentiqué a este usuario, estos son sus atributos"

OAuth — Autorización / acceso delegado

- Emite access tokens con **scopes** (`read_contacts`, `write_calendar`)
- Motor de "Log in with Google" (Hardt, 2012)
- Otorga permisos, no prueba identidad → de ahí nace OIDC como capa de identidad (OpenID Foundation, 2023)

Limitación 1 — Los scopes son rígidos y estáticos

- Primitivo central de OAuth: el "**scope**" (p. ej. `database:read`)
- Demasiado amplio para necesidades dinámicas y específicas de tarea de un agente
- Ejemplo: `database:read` estático sobre-privilegia al agente financiero que solo necesita minutos de acceso a tablas puntuales
- Token válido 1h+ = ventana de riesgo innecesaria si el agente es comprometido
- Intentar granularidad extrema → "**scope explosion**": miles de scopes hiper-específicos, inmanejable (Huang et al., 2025b; Shaikh et al., 2015)

Limitación 2 — OAuth/SAML asumen un humano presente

- Authorization Code Flow: redirige a login + pantalla de consentimiento ("Allow")
- Agentes de IA no pueden hacer clic ni consentir pop-ups (Corsha, 2023)
- **Client Credentials Flow** (M2M, no interactivo) también falla:
 - **Secret sprawl:** ¿dónde vive el `client_secret` de miles de agentes efímeros? Superficie de ataque masiva (Help Net Security, 2025a, 2025b)
 - **Sin contexto de delegación:** no puede probar que Agent-A actúa en nombre de Task-123, iniciada por User-Jane

Limitación 3 — "Autenticar una vez, confiar" es demasiado lento

- Modelos IAM tradicionales: confianza basada en sesión, válida hasta expiración del token
- Insuficiente para agentes con comportamiento potencialmente impredecible (Huang, 2025)
- Un agente legítimo puede ser secuestrado a mitad de tarea (p. ej. prompt injection)
- El contexto operativo puede cambiar (aumento de nivel de amenaza) y exigir reevaluación inmediata
- Esperar la expiración del token (minutos u horas) es demasiado lento para revocar (Help Net Security, 2025a, 2025b)

Limitación 4 — Falta contexto rico y verificable

- SAML assertions y tokens OAuth (sobre todo opacos, no-JWT) transportan poca información
- JWT permite algo más de contexto, pero los protocolos no tienen mecanismos nativos para contexto en tiempo real (IETF, 2021)
- Una decisión segura requiere responder:
 - ¿Cuál es la meta inmediata del agente?
 - ¿Cuál es la postura de seguridad de su infraestructura?
 - ¿Su comportamiento actual es anómalo vs. su línea base?
 - ¿Cuál es la sensibilidad del dato que intenta acceder ahora mismo?
- Conclusión: desajuste fundamental entre IAM centrado en humanos y agentes autónomos

3.3 El problema de bootstrapping

- Al arrancar (pod, VM, función serverless), un agente nuevo parte de **cero confianza**
- ¿Cómo obtiene de forma segura sus credenciales de identidad iniciales?
- **Enfoque legacy**: incrustar un secreto de larga vida (API key, client secret) en el entorno
- Ese secreto se vuelve un objetivo estático de alto valor
- Rotar/revocar/auditar miles de secretos para agentes efímeros = pesadilla operativa y antipatrón de seguridad

3.3 De secretos pre-colocados a identidad emitida dinámicamente

- Cambio de paradigma: **identidad de workload**, no secreto que se posee
- Principio: la identidad se deriva de hechos verificables sobre el entorno, no de un secreto
- Núcleo de Zero Trust: no confiar en lo que el workload dice ser; preguntarle a la plataforma
- **SPIFFE** (Secure Production Identity Framework for Everyone) + **SPIRE** (su implementación productiva) — estándar líder open-source (Moore et al., 2023)
- SPIFFE define el **SVID** (documento de identidad verificable) y la **Workload API** para solicitarlo

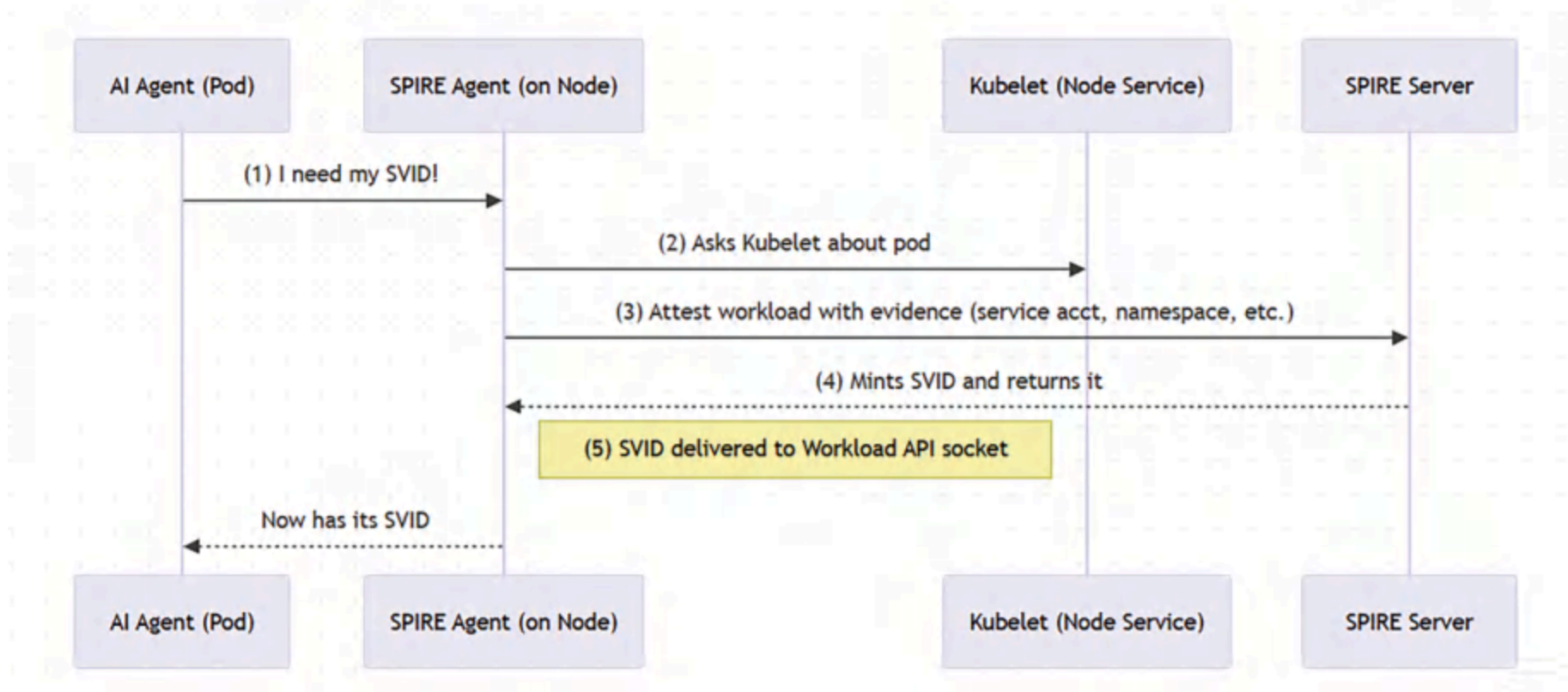
3.3 Platform attestation: la plataforma responde por el agente

En vez de que el agente presente un secreto, SPIRE consulta a la plataforma:

1. **Descubrimiento (Discovery):** el SPIRE Agent detecta el pod al arrancar en el nodo
2. **Attestation:** consulta al kubelet: service account, namespace, nodo, hash de imagen
3. **Autenticación (Authentication):** envía la evidencia al SPIRE Server, que la compara contra entradas de registro preconfiguradas
4. **Emisión (Issuance):** si coincide, el SPIRE Server emite un SVID de corta vida para esa instancia

Todo el proceso es automático, seguro y transparente para el agente (ver Fig. 3.1 en la próxima slide).

Fig. 3.1 — Flujo de attestation SPIFFE/SPIRE



Desde el desarrollador: complejidad abstraída — solo se llama a la Workload API local vía Unix Domain Socket.

Fragmento de Código 1: El Agente Obtiene su SVID (Ejemplo Conceptual en Go)

```
package main

import (
    "context"
    "log"
    "time"
    "github.com/spiffe/go-spiffe/v2/workloadapi"
)

func main() {
    ctx, cancel := context.WithTimeout(context.Background(), 3*time.Second)
    defer cancel()

    // Create a Workload API client. The socket path is automatically discovered.
    // This is the only part the agent developer needs to write.
    source, err := workloadapi.NewX509Source(ctx)
    if err != nil {
        log.Fatalf("Unable to create X509Source: %v", err)
    }
    defer source.Close()

    // The agent now has its SVID, which can be used for mTLS.
    svid, err := source.GetX509SVID()
    if err != nil {
        log.Fatalf("Unable to get X.509 SVID: %v", err)
    }

    log.Printf("Agent successfully received its identity!")
    log.Printf("SPIFFE ID: %s", svid.ID)
    log.Printf("Certificate expires at: %s", svid.Certificates[0].NotAfter)

    // Now the agent can create a TLS client or server with its SVID
    // to securely communicate with other services that trust the SPIFFE trust domain.
}
```

Sin secretos estáticos: identidad atada al entorno verificable

- El modelo resuelve el bootstrapping de forma elegante
- La identidad está atada al entorno de ejecución verificable, no a un secreto guardado
- SVIDs de **corta vida** (típicamente minutos u horas)
- **Rotación automática** antes de expirar

El SVID: la "cédula de identidad" del agente

- **SVID** (SPIFFE Verifiable Identity Document): documento criptográficamente verificable, no una simple cadena
- Formato **X.509 Certificate**: habilita mTLS, cifrado en tránsito + prueba criptográfica de identidad
- Formato **JWT**: útil para servicios sin soporte mTLS (APIs web); firmado por el trust domain
- Cada SVID contiene un **SPIFFE ID**: `spiffe://trust-domain/workload-identifier`
 - **Trust Domain**: `my-company.com` — raíz de confianza (la organización)
 - **Workload Identifier**: `/agents/financiamal-analyzer` — ruta única del agente
- El SPIFFE ID es el "quién" verificable del modelo de seguridad

3.3 → 3.4 Saber "quién" no dice "qué puede hacer"

- SPIFFE/SPIRE: identidad fuerte, dinámica, verificable — sin secretos de larga vida
- Problema de **autenticación** resuelto
- Pero esto es solo la mitad de la batalla: falta decidir qué se le permite hacer al agente

3.4 El error del "candy shell"

- Con SVID, el agente prueba quién es ante otro servicio
- Error común: detenerse ahí y asumir que el sistema ya es seguro
- Muchos equipos implementan mTLS con SVIDs y confían implícitamente puertas adentro
- Esto crea un modelo "**candy shell**": perímetro duro y cifrado, interior sin control
- **Authorization Gap**: autenticación $\neq$ autorización — probar identidad no otorga permiso de hacer nada
- Se necesita una capa robusta de autorización sobre las identidades autenticadas

3.4.1 PEP y PDP: separar aplicación de decisión

La arquitectura estándar separa **aplicación** de **decisión**:

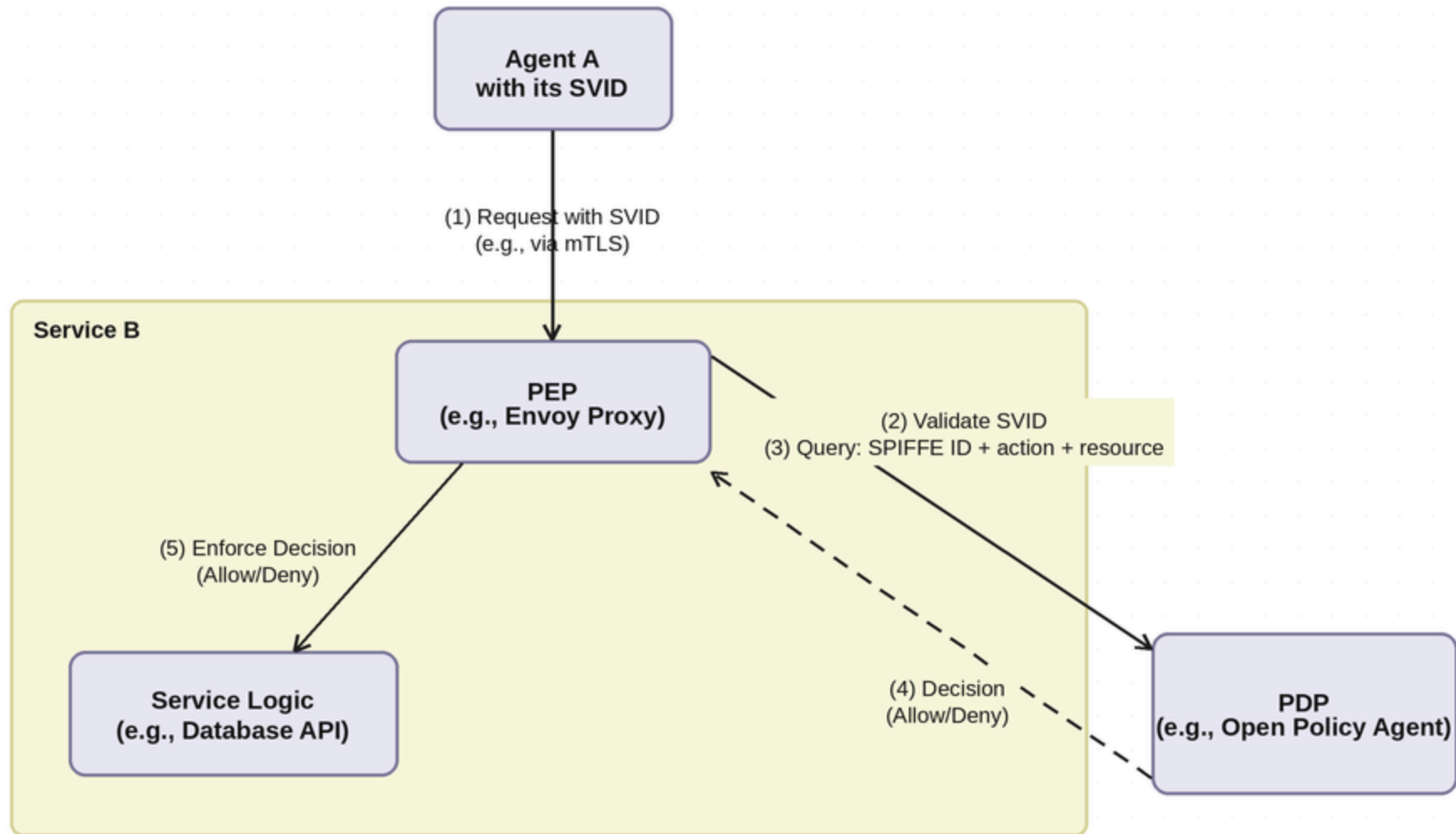
- **PEP (Policy Enforcement Point)**: frente al servicio/API; intercepta cada solicitud y aplica la decisión (no decide)
- **PDP (Policy Decision Point)**: "cerebro" centralizado; contiene las políticas y decide allow/deny
- Se alinea con DevSecOps: controles automatizados e integrados al flujo operativo (Huang et al., 2024b)
- En IA Agéntica, este modelo trabaja junto con SPIFFE

3.4.1 El flujo PEP/PDP paso a paso

1. **Solicitud:** Agent-A, con su SVID, solicita acceso a Service-B
2. **Intercepción:** el PEP de Service-B valida el SVID y extrae el SPIFFE ID
3. **Consulta:** el PEP consulta al PDP con SPIFFE ID, acción solicitada y recurso objetivo
4. **Decisión:** el PDP evalúa políticas y devuelve allow/deny
5. **Aplicación:** el PEP permite la solicitud o responde "access denied"

Desacoplamiento: los devs de servicio no codifican lógica de autorización; seguridad gestiona el PDP central (ver figura en la próxima slide).

Fig. 3.2 — Flujo de autenticación y autorización



3.4.2 Open Policy Agent y el lenguaje Rego

- **OPA (Open Policy Agent)**: implementación popular de PDP, open-source (CNCF, 2018)
- Políticas complejas expresadas en **Rego**, lenguaje declarativo de alto nivel

Rego combina programación declarativa y funcional: especifica **qué** resultado se desea, no **cómo** llegar paso a paso. Reglas describen condiciones para decisiones; el motor de OPA evalúa eficientemente.

Soporta estructuras de datos complejas, pattern matching y comprehensions — legible para devs y equipos de seguridad, capaz de navegar JSON anidado y combinar múltiples fuentes de datos.

3.4.2 De roles estáticos (RBAC) a atributos (ABAC/PBAC)

- **RBAC**: rol → permisos estáticos → lleva a "**role explosion**" al cubrir cada combinación posible
- **ABAC/PBAC** (vía OPA): decisiones en base a atributos ricos de sujeto, recurso y entorno
- El **SPIFFE ID** del agente es un atributo de sujeto crítico
- Ejemplo: el agente financiero solo debería acceder a la base de ventas durante su ventana de mantenimiento preaprobada

Fragmento de Código 2: Política ABAC de Grano Fino en Rego (para OPA)

```
package agent_authz

import rego.v1

# By default, deny all requests.
default allow := false

# Allow if the agent is the financial analyzer AND the request is for the sales API.
allow if {
    # 1. Check the agent's identity using its SPIFFE ID from the SVID.
    input.subject.spiffe_id == "spiffe://my-company.com/agents/financial-analyzer"
    # 2. Check the resource being accessed.
    input.resource.api_path == "/api/v1/sales_data"
    # 3. Check the action being performed.
    input.action.http_method == "GET"
}

# Also allow the special "auditor" agent to access anything.
allow if {
    input.subject.spiffe_id == "spiffe://my-company.com/agents/global-auditor"
}

# A more advanced rule using environmental attributes.
# Allow access only during the approved time window.
allow if {
    input.subject.spiffe_id == "spiffe://my-company.com/agents/financial-analyzer"
    input.resource.api_path == "/api/v1/sales_data"
    # Compare current time against the maintenance window ingested as external data.
    now := time.now_ns()
    now >= data.maintenance_window.start
    now < data.maintenance_window.end
}
```

ABAC combina identidad criptográfica + contexto

- Más poderosa que roles estáticos
- Usa el **SPIFFE ID** como ancla de identidad
- Lo combina con otros atributos (recurso, acción, ventana horaria) para una decisión granular y consciente del contexto

3.4.3 Progent: PEP/PDP especializado para tool use

- Riesgo primario de agentes LLM: uso (indebido) de herramientas externas
- Ataque típico: prompt injection para forzar una llamada maliciosa (p. ej. `send_email` con phishing)
- **Progent** (UC Berkeley / UC Santa Barbara): PEP/PDP especializado para tool calls (Shi et al., 2025)
- Intercepta cada llamada a herramienta y valida **no solo si puede usarla, sino los parámetros enviados**
- Ejemplo de política: "Allow `send_money` solo si `recipient_account` = 12345 y `amount` < \$100"
- Se aplica en el último momento posible, justo antes de ejecutar la herramienta

3.5 Cuatro fases del ciclo de vida de identidad

Una identidad de agente no es estática: gestionar su ciclo de vida es tan crítico como emitirla.

1. **Onboarding y Aprovisionamiento:** creación segura de la identidad inicial
2. **Operaciones en Runtime y Adaptación Dinámica:** gestión continua de credenciales, permisos y confianza
3. **Revocación:** invalidación inmediata y permanente (planeada o de emergencia)
4. **Auditoría y Forense:** análisis de la historia completa para cumplimiento e investigación

3.5.1 Fase 1: registro como código (no como secreto)

- Objetivo: pasar de proceso manual basado en secretos a uno automatizado y guiado por políticas
- Núcleo: las **entradas de registro** del SPIRE server (mapean atributos de plataforma → SPIFFE ID)
- **Registro como Código (Registration as Code)**: tratar las entradas como infraestructura crítica
 - Definidas en formato estructurado (YAML, HCL)
 - Versionadas en Git
 - Aplicadas vía pipeline CI/CD
- Resultado: rastro de auditoría completo — quién solicitó, quién aprobó (PR), cuándo se creó

Ejemplo: Una Entrada de Registro de SPIRE como Código

```
# file: financial-analyzer-agent-identity.yaml
apiVersion: spiffe.io/v1alpha1
kind: ClusterSPIFFEID
metadata:
  name: financial-analyzer-agent
spec:
  spiffeID: spiffe://my-company.com/agents/financial-analyzer
  podSelector:
    namespace: prod-agents
    labels:
      app: financial-analyzer
# This entry applies to pods with the 'financial-analyzer' label
# in the 'prod-agents' namespace.
```

Flujo: PR del ingeniero → aprobación y merge → CI/CD aplica al SPIRE server (`kubectl apply` o similar).

Attestation: el método depende de la plataforma

El modelo de attestation de SPIRE es **conectable (pluggable)**:

- **Kubernetes (`k8s_psat`)**: bound service account tokens de corta vida — namespace, service account, etiquetas del pod. A menudo la opción más segura en contenedores
- **AWS (`aws_iam`)**: prueba de rol IAM asumido en EC2/ECS
- **GCP (`gcp_iap`)**: tokens de metadata firmados de VMs y recursos GCP
- **Azure (`azure_msi`)**: endpoint de Managed Service Identity → JWT ligado a suscripción y resource group

La elección del attester define la **raíz de confianza** de la identidad del agente.

3.5.2 Fase 2: runtime — la fase más larga y compleja

- Con SVID emitido, la identidad entra en operación continua
- Credenciales y permisos deben adaptarse a condiciones cambiantes
- Identidades estáticas = frágiles; identidades dinámicas = resilientes
- **Rotación automatizada de SVIDs:** no es un "nice-to-have", aplica activamente "Assume Breach"
 - Al 50% del TTL (p. ej. 60 min), el SPIRE Agent pide un SVID nuevo automáticamente
 - El SPIRE Agent lo publica vía Workload API; casi transparente para el desarrollador
- Implicación de diseño: conexiones muy largas necesitan connection pools o reconexión periódica para tolerar la rotación

Just-in-Time (JIT) Access: privilegio bajo demanda

- Un agente orquestador puede necesitar acceso elevado **temporal** para una tarea puntual
- Otorgarlo permanentemente viola mínimo privilegio → solución: **JIT Access**

Flujo JIT

1. El agente determina que necesita acceso elevado
2. Llama al **Privilege Broker** autenticándose con su SVID base, especifica permiso y duración
3. El Broker (con su propio PDP) evalúa si puede solicitar ese permiso en ese contexto
4. Si aprueba: emite credencial de corta vida y alcance estrecho (VC, token OAuth, o política temporal)
5. El agente usa esa credencial para acceder al servicio objetivo
6. Al expirar, el acceso se revoca automáticamente

Puntaje de Confianza Dinámico (Dynamic Trust Scoring): la identidad es fija, el comportamiento no

- El SPIFFE ID es fijo; el comportamiento de un agente puede volverse errático (compromiso o bug)
- Se incorpora contexto conductual mediante un motor de **Puntaje de Confianza Dinámico (Dynamic Trust Scoring)**

Arquitectura (Fig. 3.3)

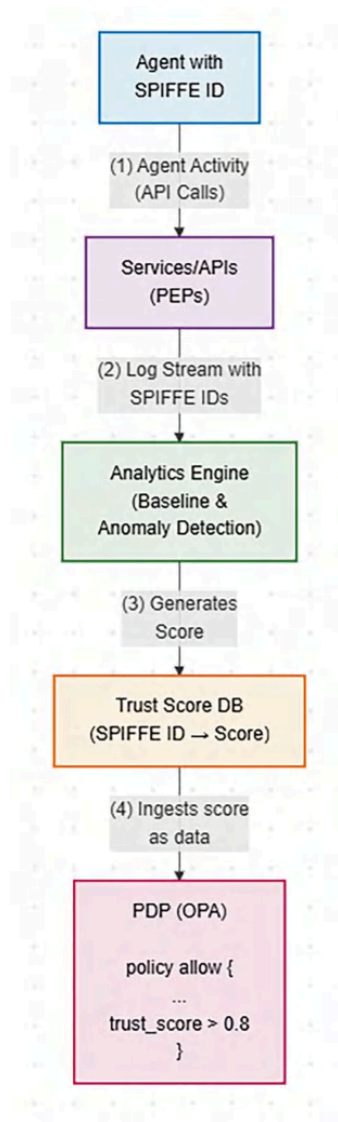
1. **Ingesta de Datos:** toda actividad (APIs, recursos, red, tools) se etiqueta con el SPIFFE ID del agente
2. **Modelado de Línea Base:** ML construye una línea base de comportamiento por agente/clase
3. **Detección de Anomalías:** compara actividad en tiempo real vs. línea base (API nueva, pico de exfiltración, horario inusual, secuencia de tools atípica)

Puntaje de Confianza Dinámico: del trust score a la política

Arquitectura (Fig. 3.3) — continuación

4. **Generación del Trust Score:** cada anomalía reduce el trust score (numérico, atado al SPIFFE ID) — 1.0 = "fully trusted", 0.2 = "highly suspicious"
5. **Integración con la Política:** el PDP (OPA) ingiere el trust score como dato adicional para sus decisiones (ver figura en la próxima slide)

Fig. 3.3 — Arquitectura de trust scoring dinámico



Política Rego consciente del trust score

Deniega acceso si el trust score del agente cae por debajo de un umbral crítico. OPA no tiene semántica *deny-overrides* incorporada: un `deny` parcial no afecta a `allow` a menos que la política lo componga explícitamente — por eso `allow` exige además `count(deny) == 0`.

```
default allow := false

# `allow` ahora exige, además de las condiciones originales, que el
# conjunto `deny` esté vacío.
allow if {
    input.subject.spiffe_id == "spiffe://my-company.com/agents/financial-analyzer"
    input.resource.api_path == "/api/v1/sales_data"
    count(deny) == 0
}

# Deniega acceso si el trust score del agente cayó por debajo del umbral crítico.
deny contains msg if {
    input.subject.spiffe_id == "spiffe://my-company.com/agents/financial-analyzer"
    data.trust_scores[input.subject.spiffe_id] < 0.5
    msg := "Access denied due to low trust score."
}
```

Verificado con `opa eval`: un agente con trust score 0.1 (muy por debajo de 0.5) da `allow = false`; el mismo agente con trust score 0.9 da `allow = true`.

3.5.3 Fase 3: revocación con múltiples capas

- Revocar rápida y completamente es crítico — no solo para emergencias, también para decomisión estándar
- Se necesitan **múltiples mecanismos** según el escenario

Mecanismo 1 — Expiración natural (decomisión planeada)

- Método más simple para un agente efímero que ya terminó su tarea
- El SVID expira en ~1h; el agente no solicita renovación porque ya no corre
- Limpio, bajo overhead, ideal para apagados planeados

Mecanismo 2 — Eliminar la entrada de registro de SPIRE (revocación demorada)

- Paso autoritativo de off-boarding: se borra la entrada del SPIRE server
- El agente sigue operando con su SVID actual hasta que expire; al intentar rotar, falla
- **Limitación:** demora hasta el TTL restante — aceptable para decomisión, no para emergencias

Mecanismo 3 — Bloqueo instantáneo vía PDP

- Para una emergencia (agente comprometido): bloquear acceso de inmediato en la capa de autorización
- Un admin de seguridad o una plataforma **SOAR** empuja una política de deny explícita y de alta prioridad
- Ejemplo (sintaxis Rego v1): `deny contains "compromised" if { input.subject.spiffe_id == "spiffe://.../compromised-agent-123" }`
- OPA carga la política en segundos: toda solicitud del agente comprometido se bloquea **aunque su SVID siga válido**
- El "freno de emergencia" del sistema

Mecanismo 4 — CAEP para terminar sesiones activas

- Las actualizaciones de política del PDP son rápidas, pero no bastan para cortar conexiones largas ya activas (p. ej. streaming)
- Estándar emergente: **CAEP** (Continuous Access Evaluation Protocol), parte de Shared Signals and Events (OpenID Foundation, 2022)

Cómo funciona

1. Un "Event Transmitter" publica un evento firmado (`{"subject": {"format": "spiffe_id", "id": "..."}, "event_type": "session-revoked"}`)
2. Todos los PEPs están suscritos como "Event Receivers"
3. Al recibir el evento, el PEP termina conexiones activas y bloquea localmente sin esperar consulta al PDP

CAEP propaga una decisión de revocación casi instantáneamente en todo el sistema distribuido.

3.5.4 Fase 4: auditoría y forense

- El rastro de auditoría persiste mucho después de que la identidad desaparece
- Esencial para cumplimiento, debugging y análisis forense post-incidente
- Clave: correlacionar eventos entre sistemas — el **SPIFFE ID** es el handle de correlación perfecto

Logs esenciales

- **SPIRE Server**: emisión/rotación de SVID, selectores que coincidieron, timestamp
- **PDP Decision Logs**: allow/deny, SPIFFE ID solicitante, recurso, acción, política disparada, timestamp
- **Application/Service Logs**: SPIFFE ID del agente llamador por cada acción de negocio

Todo consolidado en un **SIEM** central: reconstruir la historia completa de una identidad (creación, atributos, permisos, últimas acciones, comunicaciones).

3.6 Los bloques de una Zero Trust Architecture

- Identidad atestiguada (SPIFFE/SPIRE) + autorización granular (PDP/OPA) + ciclo de vida completo
- No son herramientas aisladas: son los bloques de una **Zero Trust Architecture (ZTA)**
- Zero Trust: buzzword sobreusado, pero con un núcleo simple: "**never trust, always verify**"
- Giro estratégico desde el modelo legacy "castle-and-moat" (perímetro = confianza implícita)
- En microservicios, nube y agentes autónomos, el perímetro ya no existe — el atacante puede estar dentro o fuera
- ZTA no es un producto: es una filosofía sobre 3 principios centrales

Principio 1 — Never Trust, Always Verify

- Ninguna solicitud se confía implícitamente; toda comunicación se autentica y autoriza

Cómo se implementa

- **Autenticación:** no basta con venir de una IP conocida del clúster — se exige prueba criptográfica (SVID + mTLS mutuo); la conexión solo se establece tras verificar la firma
- **Autorización:** tras autenticar, tampoco hay confianza automática — el PEP consulta al PDP para cada acción específica, cada vez
- La confianza nunca se asume: se calcula y se otorga por acción

Principio 2 — Assume Breach

- Se diseña asumiendo que los componentes **serán** comprometidos
- Objetivo: minimizar el "**blast radius**" (daño posible tras el compromiso)

Cómo se implementa

- **Ventana de robo de credenciales reducida:** SVIDs de corta vida (p. ej. 60 min) y auto-rotados vs. secretos legacy que duran hasta revocación manual (meses)
- **Poder limitado de un agente comprometido:** aunque el atacante tenga un SVID válido, el PDP solo permite el mínimo set de acciones autorizadas — sin movimiento lateral ni escalación

Principio 3: Enforce Least Privilege Access

Nota: este tercer principio no lleva numeración de sub-sección propia (no es "3.6.3"), a diferencia de los dos anteriores — se presenta igual porque es central en Zero Trust.

- Cualquier entidad —usuario o agente— recibe únicamente el mínimo permiso necesario para su tarea

Cómo se implementa

- Beneficio directo de **ABAC** con un PDP como OPA
- En vez de un rol amplio (`financial-service-role`), políticas combinan atributos específicos
- `allow { subject.spiffe_id == "..." && action == "GET" && resource == "..." }` = definición misma de mínimo privilegio
- Sin privilegios permanentes que puedan abusarse para otros fines

Tabla 3.1 — Mapeo de los principios de Zero Trust a una pila práctica de identidad de agentes

Principio de Zero Trust	Cómo se implementa
Never trust, always Verify	Autenticación: cada conexión de un agente se verifica vía mTLS usando un SVID de SPIFFE. Autorización: cada acción de un agente es autorizada explícitamente por un PDP (como OPA) para cada solicitud.
Assume breach	Seguridad de credenciales: los SVIDs de corta vida y auto-rotados de SPIRE minimizan la ventana de oportunidad para el robo de credenciales. Reducción del blast radius: políticas granulares limitan lo que puede hacer un agente comprometido.
Enforce least privilege	Modelo de políticas: usa control de acceso basado en atributos (ABAC/PBAC) en el PDP para otorgar el mínimo de permisos absolutamente necesario para una tarea dada, en base al SPIFFE ID verificado del agente y otro contexto.

Esta base permite pasar de un modelo frágil basado en perímetro a uno resiliente centrado en identidad.

3.7 Más allá de un solo trust domain

- SPIFFE/SPIRE: base poderosa dentro de una organización / trust domain — el SPIRE server es la raíz de confianza
- IA Agéntica evolucionará hacia un ecosistema descentralizado: agentes de distintas organizaciones y plataformas interactuando entre sí
- Esto exige mirar más allá de un único trust domain: interoperabilidad universal y privacidad

3.7.1 DIDs: identidad auto-soberana

- Estándares del **W3C** como alternativa a un identity provider centralizado (tipo SPIRE server)
- **DID (Decentralized Identifier)**: identificador único global, sin autoridad central (W3C, 2022)
- ID "auto-soberano" que el agente (o su dueño) crea y controla: `did:example:123456789abcdefghi`
- Un DID **resuelve** a un **DID Document** (JSON con claves públicas + endpoints de servicio)
- Cualquiera puede verificar la firma del agente sin consultar un registro central
- Ideal para ecosistemas abiertos y multi-proveedor sin una empresa controladora

3.7.1 VCs: la "estampa de visa" dentro del DID

- **Verifiable Credential (VC):** credencial digital a prueba de manipulación con afirmaciones (claims) de un emisor sobre un sujeto
- Analogía: si el DID es el número de pasaporte, la VC es la visa

Ejemplos

- Un cloud provider emite: *"Este agente pasó nuestro security scan"*
- Un consorcio de industria emite: *"Certificado para manejar datos financieros"*
- El controlador del agente emite: *"Autorizado a gastar hasta \$X"*
- Un agente acumula VCs de varios emisores y las presenta a otros servicios, que verifican la firma y deciden confiar (W3C, 2024)

3.7.2 Agent Name Service (ANS): "DNS para agentes"

- Con miles de agentes de distintas organizaciones: ¿cómo se encuentran entre sí?
- DNS da una IP, pero no dice **qué puede hacer** un agente
- Propuesta de investigación: **ANS (Agent Name Service)** — directorio universal consciente de capacidades (Huang et al., 2025a)
- Consulta por función, no por nombre: *"Find me an agent certified for legal document translation, German language"*
- Devuelve DIDs de agentes que reclaman verificablemente esas capacidades
- Concepto aún emergente — área crítica para comunicación abierta agente-a-agente

Primitivas criptográficas avanzadas

- **Digital Signatures:** base de la confianza — verifica remitente y no-alteración (p. ej. de una VC)
- **Mutual TLS (mTLS):** cliente y servidor se autentican mutuamente antes de cifrar el canal; los SVIDs de SPIFFE están diseñados para esto
- **Zero-Knowledge Proofs (ZKPs):** probar que una afirmación es verdadera sin revelar más información que su validez (Goldwasser et al., 1989)

Ejemplo práctico de ZKP: un agente prueba que posee una VC válida con el código de proyecto correcto **sin revelar el código** — autorización preservadora de privacidad para sistemas multi-parte

DIDs, VCs, ANS y ZKPs representan la frontera de la identidad de agentes.

3.8 IAM tradicional es peligrosamente inadecuado

- IA Agéntica exige un cambio equivalente en cómo se gestiona la identidad
- El IAM centrado en humanos no es solo inconveniente: es peligrosamente inadecuado para agentes dinámicos, autónomos y efímeros
- Adaptar sistemas legacy invita a fallas de seguridad
- El camino: principios y prácticas contruidos desde cero para identidades de máquina
- Punto de partida: **SPIFFE/SPIRE** — identidades de corta vida, auto-rotadas, criptográficamente verificables, sin secretos estáticos

3.8 De la autenticación a la gobernanza completa

- La autenticación es solo el comienzo
- Se extiende con autorización dinámica: PDP (OPA) desacopla decisión de aplicación
- Habilita controles granulares basados en atributos, alineados al contexto real del agente, aplicando mínimo privilegio siempre
- Toda la pila se gestiona con un **ciclo de vida completo**: onboarding automatizado + políticas como código → revocación en tiempo real → auditoría integral
- **Identidad fuerte + acceso granular + ciclo de vida completo** = pilares de una Zero Trust Architecture para IA Agéntica

Referencias citadas en el tema (1/2)

- Cantor, S., Kemp, J., Philpott, R., & Maler, E. (Eds.). (2005). *Assertions and protocols for the OASIS Security Assertion Markup Language (SAML) V2.0*. OASIS Standard.
- Cloud Native Computing Foundation. (2018). *Open Policy Agent joins Cloud Native Computing Foundation as sandbox project*. CNCF.
- Goldwasser, S., Micali, S., & Rackoff, C. (1989). The knowledge complexity of interactive proof systems. *SIAM Journal on Computing*, 18(1), 186–208.
- Huang, K., Narajala, V. S., Habler, I., & Sheriff, A. (2025). *Agent Name Service (ANS): A universal directory for secure AI agent discovery and interoperability*. arXiv.
- Huang, K., Narajala, V. S., Yeoh, J., Ross, J., Raskar, R., Harkati, Y., Huang, J., Habler, I., & Hughes, C. (2025). *A novel zero-trust identity framework for agentic AI: Decentralized authentication and fine-grained access control*. arXiv.
- Huang, K., Manral, V., & Wang, W. (2024). From LLMOps to DevSecOps for GenAI. En *Generative AI security: Theories and practices* (pp. 241–269). Springer.
- Huang, K., Yeoh, J., Wright, S., & Wang, H. (2024). Build your security program for GenAI. En *Generative AI security: Theories and practices* (pp. 99–132). Springer.

Referencias citadas en el tema (2/2)

- Internet Engineering Task Force. (2021). *RFC 9068: JSON Web Token (JWT) profile for OAuth 2.0 access tokens*. IETF.
- OpenID Foundation. (2014, actualizado 2023). *OpenID Connect Core 1.0 incorporating errata set 2*. OpenID Foundation.
- OpenID Foundation. (2022). *Shared Signals Framework — Continuous Access Evaluation Protocol (CAEP)*. OpenID Foundation.
- Shi, Z., et al. (2025). *Progent: Programmable privilege control for LLM agents*. arXiv.
- W3C. (2022). *Decentralized Identifiers (DIDs) v1.0*. W3C Recommendation.
- W3C. (2024). *Verifiable Credentials Data Model v2.0*. W3C.

Nota: tres citas del cuerpo del tema (Huang et al., 2025b y Shaikh et al., 2015, sobre "scope explosion"; Moore et al., 2023, sobre SPIFFE/SPIRE) remiten a fuentes secundarias del texto original (Huang & Hughes, 2025) que no fue posible verificar de forma independiente contra el paper/tema exacto — pendiente de confirmar contra la bibliografía completa de la fuente original.